



POLITECNICO DI BARI

**DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE
LAUREA MAGISTRALE IN INGEGNERIA DELL'AUTOMAZIONE**

Embedded control

Prof. Luca De Cicco

Title:

Open-Loop Axis Control for 3D Printer

Student:

Domenico D'Introno

ACADEMIC YEAR 2024-2025

Contents

1	Objectives of the Project	2
2	Selected Hardware	3
3	Design Choices	7
4	Firmware	8
4.1	Stepper Motor Control Module	8
4.2	Command Module for Stepper Motor Control	9
4.3	PID Controller Module	10
4.4	TOF Sensor Module	11
4.5	Timers Configuration	11
4.5.1	TIM2 – PWM Signal Generation	12
4.5.2	TIM7 – Control Loop Timing	12
4.6	UART Configuration and Reception Handler	12
4.7	I ² C Peripheral Configuration for TOF Sensor Communication	13
4.8	Main	13
5	Variables Tracing	15
6	Demonstration Video	18
7	Conclusion	19

1 Objectives of the Project

The goal of this project is to design and implement a closed-loop embedded position control system simulating the functioning of a single axis of a 3D printer. The system is capable of:

- Decoding **G-code commands** received via UART communication to change the position set-point and PID controller parameters;
- Measuring the current position with a **Time-of-Flight sensor**;
- Controlling the **linear position** via a mechanical transmission actuated by a stepper motor, whose speed is computed by a **PID controller** based on the sensed position error;
- Filtering sensor outliers and noise using a thresholding strategy.

2 Selected Hardware

The hardware components selected for this project are:

- **STM32F446RE Microcontroller (Nucleo board):** Executes control logic and manages peripherals;

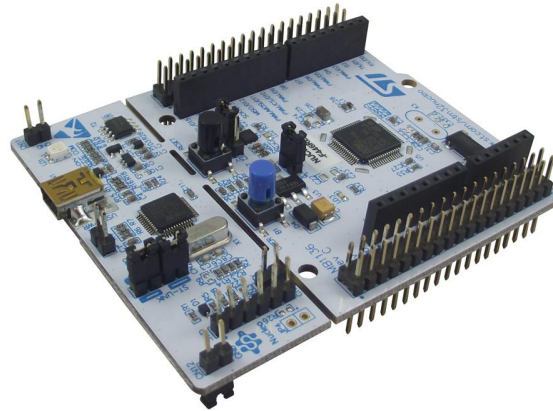


Figure 1: STM32F446RE Microcontroller

- **Nema 17 Stepper Motor:** Provides precise motion to actuate the system;

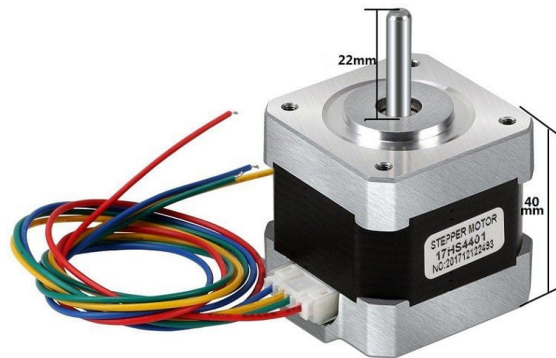


Figure 2: Nema 17 Stepper Motor

- **A4988 Stepper Driver:** Converts logic-level signals into motor drive signals;

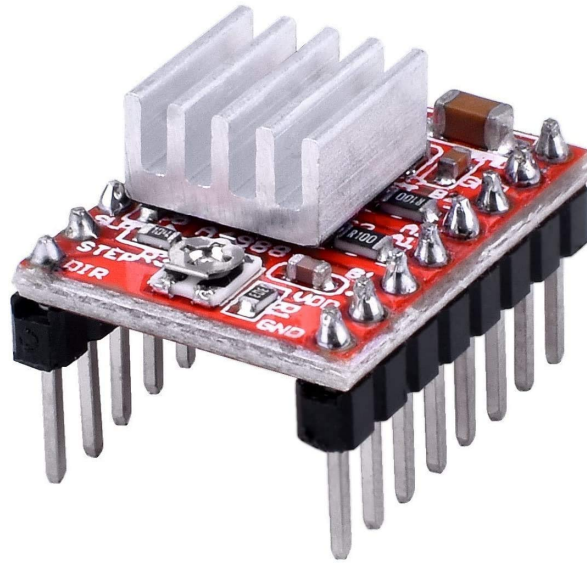


Figure 3: A4988 Stepper Driver

- **VL53L0X time-of-flight sensor:** provides a position measure thanks to the laser time-of-flight technology, that estimates the position of an object using the time that occurs from light emission to light reception.

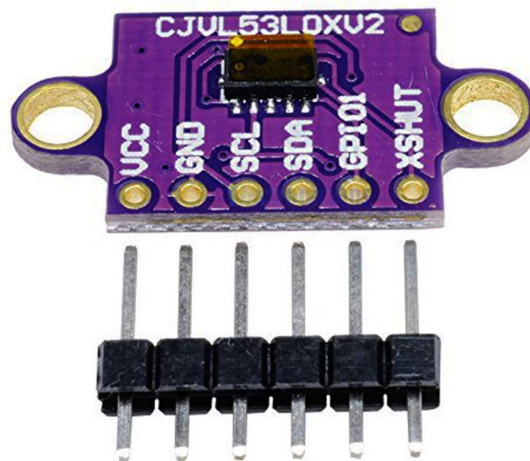


Figure 4: VL53L0X time-of-flight sensor

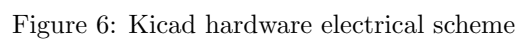
- **Pulley and Belt Mechanism:** Translates motor rotation into linear movement;



Figure 5: Pulley and Belt Mechanism

- **Wooden Frame:** Provides structural support for the physical setup.

The described hardware components have been mounted following the electrical scheme in fig. 6, using jumpwires and a breadboard.



3 Design Choices

High-end printers use **closed-loop systems**, where sensors (like encoders) continuously measure the actual position of the motors. This allows the system to detect and correct missed steps: unlike open-loop systems, the controller knows if the motor has actually moved to the desired position.

In the project, a **time-of-flight sensor** has been used to provide the position measure and a **PID controller** updates the speed of the motor based on the position error.

The user interacts with the machine by sending commands via a terminal, using **UART communication**. These commands follow the **G-code** format and allow the user to change the position of the extruder as well as set the speed and acceleration of the movement, not directly, but by changing the gains of the controller. The commands, are processed into the UART 'receive complete' handler, which parses, identifies and executes the commands. The user can specify two distinct sets of PID parameters, that can be used to differentiate traveling and extrusion movements.

Differently from the open-loop version, there is no need to implement smooth speed profiles, since the feedback system can acknowledge step loss. However, a smoother speed profile can be obtained, for instance, by lowering the proportional gain and increasing the integral gain of the controller.

In order to limit the motor reachable speed, the controller also implements some **saturation limits**.

Due to the poor precision of the sensor used, the closed loop system showed some oscillations around the setpoint. For this reason a **control dead-zone** has been implemented: any control action smaller than a user-defined threshold is neglected, ensuring having a stable extruder on the setpoint.

The whole system can be considered **interrupt-driven**, since the whole code resides into interrupt handlers and the `while(1)` section is empty.

The firmware design relies on a modular and abstracted architecture, built around custom libraries **stepper.c** and **command.c**. These components encapsulate the low-level implementation details of motor control and command parsing, respectively, allowing for cleaner and more maintainable code. By separating functionality into distinct modules, the firmware becomes easier to extend and debug. This structure also promotes reusability and improves readability, enabling a clearer distinction between hardware interaction and application logic.

The **tof_sensor.c** library encapsulates the low-level details of I2C communication with the Time-of-Flight sensor, providing a high-level interface for initialization, distance measurement, and filtering of raw data. This abstraction simplifies the integration of the sensor into the system and decouples the application logic from hardware-specific implementation.

Similarly, **pid_controller.c** implements a generic, reusable PID controller. The library allows the configuration of control parameters (K_p , K_i , K_d), definition of output limits and thresholds, and computation of corrective actions in response to the measured error. By encapsulating the control logic, it facilitates easier maintenance and flexible tuning.

4 Firmware

This section describes the firmware developed for the control of the stepper motor. The code, implemented using STM32 HAL libraries, includes the definition of libraries modules, timers configuration, the communication and the control logic.

4.1 Stepper Motor Control Module

This C module defines a library for controlling a stepper motor using an STM32 microcontroller. The library provides an interface to manage PWM signals, motor direction, microstepping configuration, acceleration and speed, using hardware peripherals such as timers and GPIO ports.

Constants and Includes

- `#define clk_freq 84000000`
Defines the clock frequency used for speed calculations.
- `#include "stepper.h"`
Includes the header file containing the **Stepper** structure and enumeration definitions.

Stepper_Init

Initializes the **Stepper** structure by assigning GPIO ports and pins, the timer handle, the distance per step, and the default acceleration. The motor is disabled by default.

Stepper_Enable

Enables the motor driver by setting the enable pin low (active-low logic).

Stepper_Disable

Disables the motor driver by setting the enable pin high.

Stepper_Start

Resets speed to zero, then starts the PWM timer that pulses the motor and the secondary timer (e.g., TIM6) used for handling acceleration via interrupts.

Stepper_Stop

Stops both the PWM signal and the acceleration timer interrupt, effectively halting the motor.

Stepper_ToggleDirection

Toggles the rotation direction of the motor by inverting the state of the direction pin.

Stepper_SetDirection

Sets the rotation direction explicitly using a **Direction** enum parameter.

Stepper_SetMicroStep

Configures the microstepping mode (e.g., full, half, quarter, eighth, or sixteenth step) by setting the MS1, MS2, and MS3 pins accordingly. The function accepts a **MicrosteppingMode** enum parameter.

Stepper_SetSpeed

Sets the motor speed by updating the PWM timer's ARR register. The command assigns a speed in mm/s, computed using the clock frequency, distance per step and desired speed. The timer counter is reset to avoid inconsistencies.

Having the `stepDist` parameter, whose value describes the covered linear distance in mm for one step, and the desired speed in mm/s in the `speed` parameter, the number of steps per second can be computed:

$$stepPerSecond = \frac{speed}{stepDist}$$

`stepPerSecond` represents basically the frequency of the PMW pulses in order to move the motor at the desired speed and its inverse is the counting period of the PWM timer.

To set this counting period, having fixed PSC value, ARR must be:

$$ARR = \frac{f_{clk}}{stepPerSecond(PSC + 1)} - 1$$

Where $f_{clk} = 84$ MHz is the frequency of the clock that feeds the timer. In this case, indeed, TIM2 is connected to APB1.

Stepper_SpeedUp

Increases the current motor speed by a given delta value. This internally calls `Stepper_SetSpeed`. This command can be used to implement acceleration in some different design logic.

4.2 Command Module for Stepper Motor Control

The `command.c` module implements a command parser and executor for controlling a stepper motor system via standard G-code and M-code commands. It communicates through UART and allows parsing, interpreting, and executing commands to modify the PID parameters or set new position targets.

Command_Init

This function initializes a `Command` structure by associating it with a `PID_Controller` and a UART interface. It also stores two sets of PID gains and thresholds, allowing dynamic switching between them during execution.

Command_Parse

Parses a string formatted as a G-code/M-code command (e.g., `G0 X100`), extracting the command type, number, flag type and flag number, through which the desired action is specified.

Command_Clear

Clears the current command structure fields and resets the input buffer to prepare for a new command.

Command_Execute

Executes the parsed command by calling the corresponding handler function. If the command is not recognized, an error message is sent over UART.

Command_G0

Sends a new setpoint to the PID controller using the first set of gains (K_{p0} , K_{i0} , K_{d0}).

Command_G1

Similar to G0, but uses the second set of PID gains (K_{p1} , K_{i1} , K_{d1}), allowing comparison between different tuning configurations.

Command_M203

Modifies the proportional gain (K_p) for either the G0 or G1 configuration. The gain is received as a percentage and converted with an empirical scaling factor.

Command_M204

Modifies the integral gain (K_i) for either the G0 or G1 configuration. The gain is received as a percentage and converted with an empirical scaling factor.

Command_M205

Modifies the derivative gain (K_d) for either the G0 or G1 configuration. The gain is received as a percentage and converted with an empirical scaling factor.

Command_T0

Updates the error threshold for the selected gain profile. This value determines how close the system must be to the target before considering the movement complete.

4.3 PID Controller Module

The module `pid_controller.c` implements a discrete-time PID (Proportional-Integral-Derivative) controller for closed-loop control of the system. It supports real-time tuning of PID parameters and threshold settings.

PID_Init

Initializes the PID controller structure by setting the gains, limits, and initial values for setpoint, integral, and error. Also sets a threshold value for ignoring small control actions.

PID_Compute

Computes the control signal based on the error between the setpoint and the measured value. It calculates the proportional, integral, and derivative contributions and returns a saturated output. If the output is within the threshold range, it is set to zero to reduce unnecessary actuation.

PID_UpdateSetpoint

Updates the target setpoint of the PID controller.

PID_UpdateProportional

Updates the proportional gain (K_p) of the controller.

PID_UpdateIntegral

Updates the integral gain (K_i) of the controller.

PID_UpdateDerivative

Updates the derivative gain (K_d) of the controller.

PID_UpdateThreshold

Updates the control output threshold, below which the controller output is ignored to prevent jitter or unnecessary micro-adjustments.

4.4 TOF Sensor Module

The module `tof_sensor.c` provides an interface to the VL53L0X Time-of-Flight sensor for distance measurement. It uses functions from the STM-provided VL53L0X sensor library, which is included via:

```
#include "vl53l0x_api.h"
#include "vl53l0x_platform.h"
```

TOF_Init

Initializes the VL53L0X sensor by performing all necessary calibration steps including static initialization, SPAD management, reference calibration, and offset calibration based on a known distance. Sets the sensor to continuous ranging mode with a predefined timing budget.

TOF_GetDistance

Returns a raw distance measurement from the sensor in millimeters. This function does not perform any filtering or validation.

TOF_GetFilteredDistance

Performs a filtered distance measurement. It returns `-1` if the value is outside a specified valid range. Otherwise, it returns the valid measured distance. This helps reduce the impact of sensor outliers in closed-loop control.

4.5 Timers Configuration

In the closed-loop control architecture, two hardware timers (TIM2 and TIM7) are used to drive and regulate the stepper motor based on real-time distance feedback from the TOF sensor.

TIM2 is configured to generate the PWM signal required to drive the motor at a variable speed. **TIM7** is configured as a periodic interrupt generator that serves as the control loop timer. At each interrupt, the system samples the TOF sensor, computes the control effort using a PID controller, and adjusts the motor speed and direction accordingly.

This section details the configuration and role of each timer.

4.5.1 TIM2 – PWM Signal Generation

TIM2 is responsible for generating the PWM signal required to drive the stepper motor. It operates in PWM mode (PWM1) and outputs the signal on pin PA0 (TIM2_CH1) that is electrically connected to the A4988 driver STEP pin. The speed of the motor is regulated by dynamically updating the timer's auto-reload value in response to the output of the PID controller. The configuration is as follows:

- **Prescaler:** 0 (maximum resolution)
- **Period:** initialized to 839999 (with an 84 MHz clock, produces a 100 Hz PWM signal), this parameter is changed at run-time by the `Stepper_SetSpeed` function to obtain a desired speed.
- **Pulse:** 10000 this value guarantees consistent duty cycles across all allowed speed interval.
- **Output Mode:** PWM1
- **Clock Source:** Internal
- **Master Mode:** Enabled (TIM_TRGO_UPDATE) – may be used to trigger slave peripherals

This timer is not linked to any step-counting mechanism in this architecture. Instead, the PID controller continuously adjusts the speed to minimize position error, and TIM2 simply reflects the current target speed through its PWM output.

4.5.2 TIM7 – Control Loop Timing

TIM7 is used to implement the control loop that periodically adjusts the stepper motor's behavior based on feedback. It generates interrupts every 10 ms to execute the control logic, which includes reading the sensor, computing the PID output, and commanding motor speed and direction.

- **Prescaler:** 8399
- **Period:** 99 (results in a 10 ms interval at 84 MHz)
- **Counter Mode:** Up

The interrupt handler (`HAL_TIM_PeriodElapsedCallback`) performs the following actions:

- Samples the TOF distance using `TOF_GetFilteredDistance()` (returns `-1` if distance is invalid).
- Rejects out-of-bound values.
- Calculates a new speed value with the `PID_Compute()` function.
- Sets motor direction depending on the sign of the PID output.
- Applies the absolute speed using `Stepper_SetSpeed()`.

4.6 UART Configuration and Reception Handler

The UART communication is used to receive commands and to send log data and error messages through the serial terminal Putty. In this project, **USART2** is used and it is configured with the following parameters:

- **Baud rate:** 115200 bps
- **Word length:** 8 bits
- **Stop bits:** 1

- **Parity:** None
- **Mode:** TX/RX (transmit and receive)
- **Oversampling:** 16

The reception of UART data is handled asynchronously using interrupts. The function `HAL_UART_RxCpltCallback()` is triggered whenever a byte is received. The firmware uses this mechanism to construct a command buffer, processing complete commands terminated by the `\r` character.

The interrupt handler performs the following operations:

1. Appends the received byte to the buffer `rx.data`.
2. If the received byte is a carriage return (`\r`), the full command is parsed using `Command_Parse()` and executed via `Command_Execute()`.
3. The buffer is cleared and reset using `memset()` and `Command_Clear()`.
4. The UART is re-armed to receive the next byte using `HAL_UART_Receive_IT()`.

This interrupt-driven approach ensures a responsive and non-blocking communication mechanism, ideal for command-based control interfaces.

4.7 I²C Peripheral Configuration for TOF Sensor Communication

The Time-of-Flight (TOF) distance sensor is interfaced with the STM32 microcontroller using the I²C protocol. In this project, the STM32 acts as an I²C master and communicates with a single TOF sensor configured as a slave.

The I²C peripheral is initialized using the STM32 HAL library, and all sensor read/write operations are handled by a sensor-specific driver (e.g. ST's VL53L0X API). The I²C hardware block used in this implementation is **I2C1**.

The I2C1 peripheral is configured in standard mode with a clock speed of 100 kHz, using 7-bit addressing. The SCL and SDA lines are mapped to pins PB6 and PB7, respectively, with open-drain mode and no internal pull-ups, as required by the I²C specification. External 4.7 k Ω pull-up resistors are connected to both lines.

The configured I2C1 peripheral is used by the TOF sensor driver for performing low-level read/write operations over the I²C bus. The driver internally calls STM32 HAL functions such as:

- `HAL_I2C_Mem_Read()` – to read sensor registers
- `HAL_I2C_Mem_Write()` – to write configuration data

These calls abstract the I²C communication, allowing higher-level application functions (such as `TOF_GetFilteredDistance()`) to operate without direct concern for the hardware handling.

4.8 Main

The **main.c** file serves as the central entry point of the firmware and coordinates the initialization and execution of all core components in the system. This includes the configuration of the **TOF sensor**, **stepper motor**, **PID controller** and **command interface**.

Upon startup, the system initializes all hardware peripherals, including I2C, UART, GPIO, and timers. The `TOF_Init()` function initializes the Time-of-Flight sensor over the I2C interface with a specified reference distance, allowing the device to begin acquiring accurate proximity measurements.

The stepper motor driver is configured using `Stepper_Init()`, associating it with its relevant timer and GPIO pins for stepping, direction control, and microstepping resolution. The stepping mode is then set to sixteenth-step via `Stepper_SetMicroStep()`, enabling fine-grained position control.

To regulate motion based on sensor feedback, a `PID_Controller` instance is initialized through `PID_Init()` using tuned gain parameters and output limits. The system then links the PID controller to the communication interface using `Command_Init()`, enabling runtime configuration and control over UART.

An instructional command message is sent to the user upon startup, and UART reception is enabled in interrupt mode via `HAL_UART_Receive_IT()`, supporting asynchronous command input. A timer (TIM7) is started in interrupt mode to periodically trigger control actions.

The **main loop** remains empty, as the firmware is designed to operate in a fully **interrupt-driven** fashion.

This modular structure enables efficient, low-latency control of a motorized actuator in response to proximity feedback, with dynamic reconfigurability via UART-based command inputs.

5 Variables Tracing

This section illustrates and discusses the variable tracing performed using the Serial Wire Viewer (SWV) trace technology. Also a table, showing the measured errors at different distances is shown.

Proportional Action Only

The following results were obtained by setting both the integral and derivative gains to zero, thereby isolating the proportional contribution of the controller.

Figure 7 displays the proportional control action, labeled as **P_out**, and the corresponding **error** signal. As expected, the two signals exhibit the same shape and differ only by a scaling factor, namely the proportional gain K_p , confirming the exclusive proportionality of the control action.

With these controller settings, the **distance** signal follows an overdamped response, as shown in Figure 8.

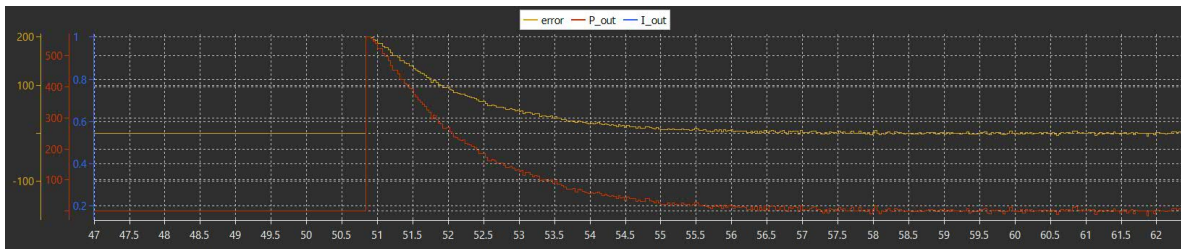


Figure 7: Proportional only controller action vs position error

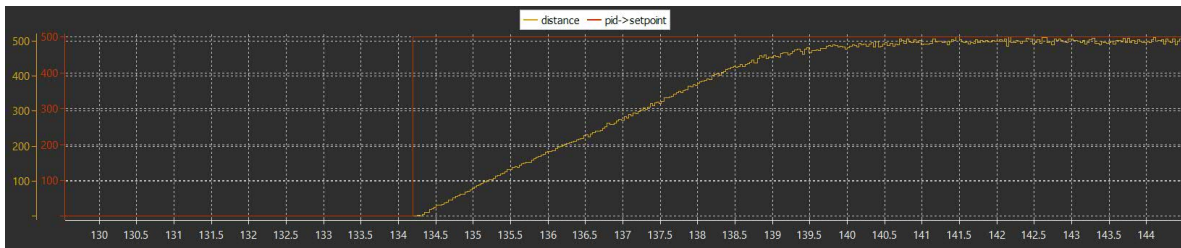


Figure 8: Proportional only setpoint following

Proportional-Integral Action

By introducing integral control through an adjustment of the K_i gain—performed using the **M204_Command** via the terminal—the resulting system exhibits an underdamped distance profile.

Figure 9 shows both the proportional and integral components of the control signal. The proportional term reacts immediately to changes in the setpoint, while the integral term accumulates over time by integrating the error. Consequently, even after the setpoint is reached, the integral component remains non-zero, causing the system to overshoot.

The integral action is then gradually discharged, eventually inverting its sign and driving the system back toward the setpoint. This behavior is evident in Figure 10, where the **setpoint** is compared against the **distance**, showing a 25% overshooting.

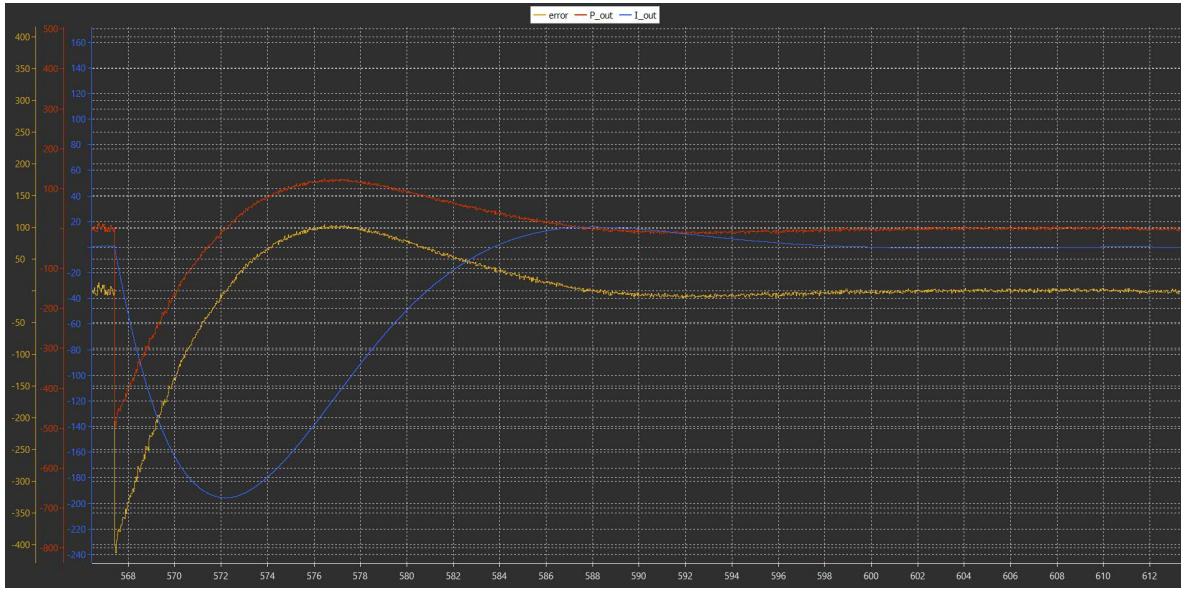


Figure 9: Proportional-Integral controller action vs position error



Figure 10: Proportional-Integral setpoint following

These tests demonstrate that using a proportional-only control action results in a lower settling time, but at the cost of a higher rise time and increased mechanical stress. This is due to the abrupt change in the control signal. Conversely, by combining proportional and integral actions with appropriately tuned controller gains, the system achieves a lower rise time and a smoother speed profile—thanks to the gradual buildup of the integral term. However, this comes at the expense of a higher overshoot.

Position Errors

The table below presents the measured positions (manually recorded) corresponding to various setpoint distances during the testing phase. It highlights how sensor accuracy degrades as the distance from the target increases, constantly underestimating the actual distance.

Reference position	Actual position	Error	Error %
100	97	3	3.0%
200	194	6	3.0%
300	286	14	4.7%
400	380	20	5.0%
500	465	35	7.0%

Table 1: Position offset at different positions in *mm*

6 Demonstration Video

The demonstration video shows:

- UART command control;
- Stepper motor movement;
- SWV (Serial Wire Viewer) real-time tracing of the measured position variable vs the user-defined position setpoint.

Link: <https://youtu.be/x8zu2CGWowc>

7 Conclusion

This project demonstrates a working closed-loop PID-controlled motion system using STM32 microcontroller. It effectively integrates feedback sensing, motion control, UART communication, and real-time debugging. The architecture is modular and forms a solid foundation for future multi-axis motion systems.